

Task 1 — Configure Git Username

Command:

```
git config --global user.name "Your Name"
```

Description:

Sets the username that Git uses to identify the author of commits.

Task 2 — Configure Git Email

Command:

```
git config --global user.email "your-email@example.com"
```

Description:

Sets the email address associated with Git commits.

Task 3 — Verify Git Configuration

Command:

```
git config --global --list
```

Description:

Displays the Git configuration settings, including the configured username and email.

Task 4 — Check Git Installation

Command:

```
git --version
```

Description:

Displays the installed Git version and verifies that Git is available on the system.

Part A — Git Setup & Repository

Task 5 — Create the Employee Management Project Folder

Command:

```
mkdir employee-management
```

Description:

Creates a new folder named employee-management for the project.

Task 6 — Enter the Project Folder

Command:

```
cd employee-management
```

Description:

Moves the terminal into the employee-management project directory.

Task 7 — Initialize the Git Repository

Command:

```
git init
```

Description:

Initializes the project folder as a Git repository.

Task 8 — Create Project Files

Command:

```
touch index.html employees.html README.md .gitignore
```

Description:

Creates the required project files: index.html, employees.html, README.md, and .gitignore.

Task 9 — Check Repository Status

Command:

`git status`

Description:

Displays the current state of the repository and shows untracked or modified files.

Task 10 — Stage All Project Files

Command:

`git add .`

Description:

Stages all eligible project files and changes for the next commit.

Task 11 — Create the First Commit

Command:

`git commit -m "Initial project setup"`

Description:

Creates the first commit and records the initial project setup in Git history.

Task 12 — View Short Commit History

Command:

`git log --oneline`

Description:

Displays the commit history in a short, one-line format.

Part B — GitHub Repository

Task 13 — Connect Local Repository to GitHub

Command:

`git remote add origin repository-url`

Description:

Connects the local Git repository to the GitHub repository using the remote name origin.

Task 14 — Rename Default Branch to Main

Command:

`git branch -M main`

Description:

Renames the current branch to main.

Task 15 — Verify the Remote Repository

Command:

`git remote -v`

Description:

Displays the configured GitHub remote URL for fetching and pushing.

Task 16 — Push Main Branch to GitHub

Command:

`git push -u origin main`

Description:

Uploads the local main branch to GitHub and establishes the upstream relationship.

Task 17 — Display Detailed Remote Information

Command:

`git remote show origin`

Description:

Displays detailed information about the configured remote repository and its branches.

Task 18 — Understand Local and Remote Branches

Commands:

`git branch`

`git branch -r`

Description:

`git branch` displays local branches, while `git branch -r` displays remote branches.

Part C — .gitignore

Task 19 — Create and Configure .gitignore

Command:

`git status`

Description:

Checks the repository status after configuring .gitignore.

Required ignored items: .env, node_modules/, .log files, and .DS_Store.

Task 20 — Create a Test .env File

Command:

`touch .env`

Description:

Creates a test .env file to verify whether Git ignores it.

Task 21 — Verify Ignored Files

Command:

`git status --ignored`

Description:

Displays files ignored by Git and verifies that .env is being ignored.

Task 22 — Understand Why .env Should Not Be Committed

Command:

`git status`

Description:

Used to verify that the .env file is not being tracked.

.env files commonly contain passwords, API keys, access tokens, and other sensitive information. Therefore, they should generally not be committed to GitHub.

Part D — Branching

Task 23 — Create Employee List Branch

Command:

`git branch feature/employee-list`

Description:

Creates a new feature branch named feature/employee-list.

Task 24 — Switch to Employee List Branch

Command:

`git switch feature/employee-list`

Description:

Switches the current working branch to feature/employee-list.

Task 25 — Verify Current Branch

Command:

`git branch`

Description:

Lists local branches and identifies the current branch with an asterisk.

Task 26 — Check Changes Before Commit

Command:

`git status`

Description:

Checks the changes made to the employee listing before staging and committing them.

Task 27 — Stage Employee Listing Changes

Command:

`git add employees.html`

Description:

Stages the changes made to employees.html.

Task 28 — Commit Employee Listing Feature

Command:

`git commit -m "Add employee listing table"`

Description:

Creates a commit containing the employee listing feature.

Task 29 — Push Employee List Branch to GitHub

Command:

`git push -u origin feature/employee-list`

Description:

Uploads the feature/employee-list branch to GitHub.

Task 30 — Create Employee Search Branch

Command:

`git switch -c feature/employee-search`

Description:

Creates the feature/employee-search branch and immediately switches to it.

Task 31 — Check Employee Search Changes

Command:

`git status`

Description:

Displays the changes made while developing the employee search feature.

Task 32 — Stage Employee Search Changes

Command:

`git add employees.html`

Description:

Stages the changes made to employees.html for the employee search feature.

Task 33 — Commit Employee Search Feature

Command:

```
git commit -m "Add employee search feature"
```

Description:

Creates a commit containing the employee search functionality.

Task 34 — Push Employee Search Branch to GitHub

Command:

```
git push -u origin feature/employee-search
```

Description:

Uploads the feature/employee-search branch to GitHub.

Task 35 — Display All Local and Remote Branches

Command:

```
git branch -a
```

Description:

Displays all local branches and remote branches together.

Part E — Git Branch Visualization

Task 36 — Display Git History as a Graph

Command:

```
git log --oneline --graph --all --decorate
```

Description:

Displays the Git commit history in a graphical tree format, including all branches and their branch names.

Task 37 — Identify the Latest Commit

Command:

```
git log -1
```

Description:

Displays information about the latest commit in the current branch.

Task 38 — Identify the Latest Commit Across All Branches

Command:

```
git log --oneline --all --decorate --graph
```

Description:

Displays the commit history of all branches in graph format and helps identify which branch contains the latest commit.

Additional Useful Git Commands

Task 39 — View Unstaged Changes

Command:

```
git diff
```

Description:

Displays changes that have been made but have not yet been staged.

Task 40 — View Staged Changes

Command:

```
git diff --staged
```

Description:

Displays changes that have already been staged and are ready for commit.

Task 41 — View Latest Commit Details

Command:

`git show`

Description:

Displays detailed information about the latest commit and the changes included in it.

Task 42 — View Remote Branches

Command:

`git branch -r`

Description:

Displays only the branches available on the remote repository.

Task 43 — Fetch Remote Changes

Command:

`git fetch`

Description:

Downloads information about changes from the remote repository without automatically merging them into the current branch.

Task 44 — Pull Changes from GitHub

Command:

`git pull`

Description:

Downloads changes from the remote repository and integrates them into the current local branch.